

vulnedu.py

```
from flask import Flask, jsonify
from routes.main_routes import main_bp
from routes.api_routes import api_bp
from routes.learn_routes import learn_bp
from routes.utility_routes import utility_bp
import config
import os
import sys

def create_app():
    """Application factory pattern with enhanced debugging for Render deployment"""
    print("[Flask] Initializing VulnEdu application...")

    # Log startup information to help debug Render deployment
    print(f"[Flask] Python version: {sys.version}")
    print(f"[Flask] Current working directory: {os.getcwd()}")

    # Print environment variables related to database or Render
    print("[Flask] Environment variables:")
    for key in sorted(os.environ.keys()):
        if any(term in key for term in ['DATABASE', 'POSTGRES', 'DB_', 'RENDER', 'PORT']):
            value = os.environ[key]
            # Mask credentials in output
            if 'URL' in key or 'PASSWORD' in key or 'SECRET' in key:
                if value and len(value) > 12:
                    value = value[:8] + '...' + value[-4:]
            print(f"  {key}: {value}")

    # Ensure DATABASE_URL is properly set in environment if available in config
    if config.DATABASE_URL and not os.environ.get('DATABASE_URL'):
        print("[Flask] Re-exporting DATABASE_URL from config to environment")
        os.environ['DATABASE_URL'] = config.DATABASE_URL

    app = Flask(__name__)
    app.config['SECRET_KEY'] = config.SECRET_KEY

    # Pass database configuration to the Flask app
    app.config['DATABASE_ENABLED'] = config.DATABASE_ENABLED
    app.config['SQLALCHEMY_DATABASE_URI'] = config.SQLALCHEMY_DATABASE_URI
    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = config.SQLALCHEMY_TRACK_MODIFICATIONS

    # Print database configuration
    print(f"[Flask] Database enabled: {app.config['DATABASE_ENABLED']}")
    if app.config['DATABASE_ENABLED']:
        print(f"[Flask] Database URI: {app.config['SQLALCHEMY_DATABASE_URI'][:15]}...")

    # Register blueprints
    app.register_blueprint(main_bp)
    app.register_blueprint(api_bp, url_prefix='/api')
    app.register_blueprint(learn_bp, url_prefix='/learn')
    app.register_blueprint(utility_bp)

    # Health check endpoint for Render with enhanced debug info
    @app.route('/health')
    def health():
        # Include database status in health check
        db_status = 'enabled' if config.DATABASE_ENABLED else 'disabled'
        return jsonify({
            'status': 'healthy',
            'service': 'VulnEdu',
            'database': db_status,
            'version': '1.1.0' # Include version for tracking
        }), 200

    # Debug endpoint to show memory usage
    @app.route('/memory-usage')
    def memory_usage():
        import psutil
        process = psutil.Process(os.getpid())
        memory_info = process.memory_info()

    return jsonify({
```

```

'rss': f"{memory_info.rss / (1024 * 1024):.2f} MB",
'vms': f"{memory_info.vms / (1024 * 1024):.2f} MB",
'percent': process.memory_percent(),
'cpu_percent': process.cpu_percent(interval=1.0)
})

# Add debug route to show environment variables
@app.route('/debug')
def debug():
    # Return filtered environment variables
    env_vars = {}
    for key in sorted(os.environ.keys()):
        value = os.environ[key]
        # Mask credentials
        if 'URL' in key or 'PASSWORD' in key or 'SECRET' in key:
            if value and len(value) > 12:
                value = value[:8] + '...' + value[-4:]
        env_vars[key] = value

    # Add port information
    port = os.environ.get('PORT', 'Not set')

    return jsonify({
        'port': port,
        'env_vars': env_vars
    })

print("[Flask] VulnEdu application initialized successfully!")
return app

# Create the app instance at module level for Gunicorn
app = create_app()

# For local testing, ensure port is set correctly
if __name__ == "__main__":
    # This only runs in local development
    print("[Flask] Starting VulnEdu in DEVELOPMENT mode...")

    # Create required directories
    for directory in [config.CACHE_DIR, config.NVD_DIR, config.NVD_HISTORICAL_DIR,
config.NVD_PROCESSED_DIR]:
        os.makedirs(directory, exist_ok=True)

    # Get port from environment or use default
    port = int(os.environ.get('PORT', 5000))
    print(f"[Flask] Running on port: {port}")

    # Run with basic settings for local dev
    app.run(host='0.0.0.0', port=port, debug=True)

```